

ลำดับ $[()]$ นั้นถูกต้องตามหลักไวยากรณ์ (legal) แต่ลำดับ $[()]$ นั้นผิดพลาด เห็นได้ชัดว่ามันไม่คุ้มค่าที่จะเขียนโปรแกรมขนาดใหญ่เพื่อตรวจสอบเรื่องนี้ แต่มันกลับกลายเป็นว่าการตรวจสอบสิ่งเหล่านี้สามารถทำได้ง่ายมาก เพื่อความเรียบง่าย เราจะตรวจสอบเฉพาะความสมดุลของการจับคู่วงเล็บ (parentheses), วงเล็บเหลี่ยม (brackets), และปีกกา (braces) โดยละเว้นอักขระอื่นๆ ที่ปรากฏขึ้นมา

อัลกอริทึมอย่างง่ายจะใช้สแต็ก (stack) และมีขั้นตอนดังนี้:

สร้างสแต็กว่างเปล่าขึ้นมา อ่านอักขระไปเรื่อยๆ จนกว่าจะจบไฟล์ (end of file) หากอักขระนั้นเป็นสัญลักษณ์เปิด (opening symbol) ให้push มันลงในสแต็ก หากมันเป็นสัญลักษณ์ปิด (closing symbol) และสแต็กว่างเปล่า ให้รายงานข้อผิดพลาด (report an error) หากไม่เป็นเช่นนั้น ให้ดึงข้อมูลออกจากสแต็ก (pop) หากสัญลักษณ์ที่ถูกดึงออกมาไม่ใช่สัญลักษณ์เปิดที่เข้าคู่กัน ให้รายงานข้อผิดพลาด เมื่ออ่านจนจบไฟล์แล้ว หากสแต็กยังไม่ว่างเปล่า ให้รายงานข้อผิดพลาด

คุณน่าจะพอเห็นภาพได้ว่าอัลกอริทึมนี้สามารถทำงานได้จริง มันทำงานเป็นเส้นตรงอย่างชัดเจน (linear) และใช้การอ่านข้อมูลนำเข้าเพียงแค่รอบเดียว (one pass) ดังนั้นมันจึงประมวลผลทันที (online) และรวดเร็วมาก สามารถเพิ่มการทำงานพิเศษเพื่อตัดสินใจว่าจะทำอย่างไรเมื่อมีการรายงานข้อผิดพลาด เช่น การระบุสาเหตุที่เป็นไปได้

นิพจน์แบบโพสต์ฟิกซ์ (Postfix Expressions)

สมมติว่าเรามีเครื่องคิดเลขพกพาและต้องการคำนวณค่าใช้จ่ายในการไปซื้อของ ในการทำเช่นนั้น เราจะบวกรายการตัวเลขต่างๆ เข้าด้วยกันแล้วคูณผลลัพธ์ด้วย 1.06; นี่คือการคำนวณราคาซื้อของสินค้าโดยรวมภาษีการขายเข้าไปด้วย หากสินค้ามีราคา 4.99, 5.99, และ 6.99 วิธีที่เป็นธรรมชาติในการป้อนข้อมูลนี้คือลำดับ

$$4.99 + 5.99 + 6.99 * 1.06 =$$

ผลลัพธ์ที่ได้จะขึ้นอยู่กับเครื่องคิดเลข ซึ่งอาจจะได้คำตอบที่ตึงใจไว้คือ 19.05 หรือคำตอบทางวิทยาศาสตร์คือ 18.39 เครื่องคิดเลข 4 พังก์ชันพื้นฐานส่วนใหญ่จะให้คำตอบแรก แต่เครื่องคิดเลขวิทยาศาสตร์ขั้นสูงหลายรุ่นจะทราบดีว่าการคูณมีลำดับความสำคัญ (precedence) สูงกว่าการบวก ในทางกลับกัน สินค้าบางรายการอาจต้องเสียภาษีและบางรายการไม่ต้องเสียภาษี ดังนั้นหากมีเพียงสินค้ารายการแรกและรายการสุดท้ายเท่านั้นที่ต้องเสียภาษีจริง ลำดับ

$$4.99 * 1.06 + 5.99 + 6.99 * 1.06 =$$

จะให้คำตอบที่ถูกต้อง (18.69) บนเครื่องคิดเลขวิทยาศาสตร์ และให้คำตอบที่ผิดพลาด (19.37) บนเครื่องคิดเลขพื้นฐาน เครื่องคิดเลขวิทยาศาสตร์มักจะมียกเลิกมาให้ด้วย ดังนั้นเราจึงสามารถหาคำตอบที่ถูกต้องได้เสมอด้วยการใส่วงเล็บ แต่สำหรับเครื่องคิดเลขแบบง่าย เราจำเป็นต้องจดจำผลลัพธ์ระหว่างทางเอาไว้

ลำดับการประเมินค่าทั่วไปสำหรับตัวอย่างนี้อาจเป็นการคูณ 4.99 และ 1.06 โดยเก็บคำตอบไว้เป็น A_1 จากนั้นเราบวก 5.99 และ A_1 เก็บผลลัพธ์ไว้ใน A_1 เราคูณ 6.99 และ 1.06 เก็บคำตอบไว้ใน A_2 และเสร็จสิ้นโดยการบวก A_1 และ A_2 ทำให้ได้คำตอบสุดท้ายใน A_1 เราสามารถเขียนลำดับการดำเนินการนี้ได้ดังนี้:

$$4.99 \quad 1.06 \quad * \quad 5.99 \quad + \quad 6.99 \quad 1.06 \quad * \quad +$$

สัญกรณ์ (notation) นี้เรียกว่า โพสต์ฟิกซ์ (postfix) หรือ reverse Polish notation และจะถูกประเมินค่าตามที่เราได้อธิบายไว้ข้างต้น วิธีที่ง่ายที่สุดในการทำสิ่งนี้คือการใช้สแต็ก เมื่อเห็นตัวเลข มันจะถูกpushลงในสแต็ก; เมื่อเห็นเครื่องหมาย (operator) เครื่องหมายนั้นจะถูกนำไปใช้กับ

ตัวเลขสองตัว (สัญลักษณ์) ที่ถูกป๊อป (popped) ออกจากสแต็ก และผลลัพธ์จะถูกพusadaลงในสแต็ก ตัวอย่างเช่น นิพจน์แบบโพสต์ฟิสิกซ์

6 5 2 3 + 8 * + 3 + *

จะถูกประเมินค่าดังต่อไปนี้:

สัญลักษณ์ 4 ตัวแรกถูกวางลงในสแต็ก สแต็กที่ได้จะเป็นดังนี้

topOfStack →	3
	2
	5
	6

ถัดมา อ่านพบ '+' ดังนั้น 3 และ 2 จะถูกดึงออกจากสแต็ก และผลรวมคือ 5 จะถูกพusadaลงไป ถัดมา 8 จะถูกพusadaลงไป

topOfStack →	5
	5
	6

topOfStack →	8
	5
	5
	6

ตอนนี้ อ่านพบ '*' ดังนั้น 8 และ 5 จะถูกดึงออก และ $5 * 8 = 40$ จะถูกพusadaลงไป

topOfStack →	40
	5
	6

ถัดมา พบ '+' ดังนั้น 40 และ 5 จะถูกดึงออก และ $5 + 40 = 45$ จะถูกพusadaลงไป ตอนนี้ 3 จะถูกพusadaลงไป

topOfStack →	45
	6

topOfStack →	3
	45
	6

ถัดมา '+' จะดึง 3 และ 45 ออก และพusada $45 + 3 = 48$ ลงไป

topOfStack →	48
	6

สุดท้าย อ่านพบ '*' และ 48 กับ 6 จะถูกดึงออก; ผลลัพธ์ $6 * 48 = 288$ จะถูกพUSHลงไป

topOfStack →	288
--------------	-----

เวลาที่ใช้ในการประเมินค่านิพจน์แบบโพสต์ฟิกซ์คือ $O(N)$ เนื่องจากการประมวลผลองค์ประกอบแต่ละตัวในข้อมูลนำเข้าประกอบด้วยการทำงานของสแต็กและใช้เวลาคงที่ (constant time) อัลกอริทึมในการทำเช่นนี้จึงเรียบง่ายมาก สังเกตว่าเมื่อให้สมการในรูปแบบโพสต์ฟิกซ์มาแล้ว ไม่มีความจำเป็นต้องรู้ลำดับความสำคัญใดๆ เลย; นี่คือข้อได้เปรียบที่เห็นได้ชัด

การแปลงสมการอินฟิกซ์เป็นโพสต์ฟิกซ์ (Infix to Postfix Conversion)

สแต็กไม่เพียงแต่ใช้ประเมินค่านิพจน์แบบโพสต์ฟิกซ์ได้เท่านั้น แต่เรายังสามารถใช้สแต็กเพื่อแปลงนิพจน์ในรูปแบบมาตรฐาน (หรือที่เรียกว่าอินฟิกซ์ - infix) ไปเป็นรูปแบบโพสต์ฟิกซ์ได้อีกด้วย เราจะมุ่งเน้นไปที่ปัญหาทั่วไปในฉบับย่อโดยอนุญาตให้มีเพียงเครื่องหมาย $+$, $*$, $($, $)$ และยืนยันให้ใช้กฎลำดับความสำคัญตามปกติ เราจะตั้งสมมติฐานเพิ่มเติมว่าสมการนั้นถูกต้อง สมมติว่าเราต้องการแปลงสมการอินฟิกซ์

$$a + b * c + (d * e + f) * g$$

ไปเป็นโพสต์ฟิกซ์ คำตอบที่ถูกต้องคือ $a \ b \ c \ * \ + \ d \ e \ * \ f \ + \ g \ * \ +$

เมื่ออ่านพบตัวถูกดำเนินการ (operand) จะถูกนำไปวางไว้ที่ผลลัพธ์ทันที ส่วนตัวดำเนินการ (operators) จะยังไม่ถูกนำออกไปที่ผลลัพธ์ในทันที ดังนั้นพวกมันจึงต้องถูกเก็บไว้ที่ใดที่หนึ่ง สิ่งที่ต้อง做的就是การนำตัวดำเนินการที่พบเห็นแล้วแต่ยังไม่ได้นำไปแสดงที่ผลลัพธ์ ไปวางไว้บนสแต็ก เราจะพUSHลงสแต็กเมื่อพบเจอด้วย เราจะเริ่มต้นด้วยสแต็กที่ว่างเปล่า

หากเราพUSHลงสแต็ก เราจะทำการดึง (pop) ข้อมูลในสแต็ก โดยเขียนสัญลักษณ์ต่างๆ ออกมาจนกว่าเราจะพบกับวงเล็บเปิด (ที่คู่กัน) ซึ่งมันจะถูกดึงออกมาแต่ไม่ต้องส่งออกไปที่ผลลัพธ์

หากเราพบสัญลักษณ์อื่นใด ($+$, $*$, $)$) เราจะดึงข้อมูลออกจากสแต็กจนกว่าเราจะพบข้อมูลที่มีลำดับความสำคัญต่ำกว่า ข้อยกเว้นประการหนึ่งคือ เราจะไม่นำวงเล็บเปิด (ออกจากสแต็กเลยยกเว้นเมื่อกำลังประมวลผลวงเล็บปิด) สำหรับจุดประสงค์ของการดำเนินการนี้ $+$ มีลำดับความสำคัญต่ำสุด และ $($ มีลำดับสูงสุด เมื่อการป้อนเสร็จสิ้น เราจะพUSHตัวดำเนินการตัวนั้นลงสแต็ก

สุดท้าย หากเราอ่านจนจบข้อมูล (end of input) เราจะดึงข้อมูลในสแต็กจนกว่าจะว่างเปล่า โดยเขียนสัญลักษณ์ลงบนผลลัพธ์

แนวคิดของอัลกอริทึมนี้คือ เมื่อพบตัวดำเนินการ มันจะถูกวางลงในสแต็ก สแต็กจะทำหน้าที่เป็นตัวแทนของตัวดำเนินการที่ค้างอยู่ (pending operators) อย่างไรก็ตาม ตัวดำเนินการบางตัวในสแต็กที่มีลำดับความสำคัญสูงตอนนี้ถือว่าเสร็จสิ้นแล้วและควรถูกดึงออก เนื่องจากพวกมันจะไม่ค้างอยู่อีกต่อไป ดังนั้นก่อนที่จะวางตัวดำเนินการใหม่ลงในสแต็ก ตัวดำเนินการเดิมที่อยู่ในสแต็ก และซึ่งจำเป็นต้องดำเนินการให้เสร็จสิ้นก่อนตัวดำเนินการปัจจุบัน จะต้องถูกดึงออก สิ่งนี้ได้แสดงให้เห็นในตารางต่อไปนี้:

สมการ	สแต็กเมื่อตัวดำเนินการที่ 3 ถูกประมวลผล	การดำเนินการ
$a * b - c + d$	—	— เสร็จสมบูรณ์; + ถูกพUSH
$a/b + c * d$	+	ไม่มีอะไรเสร็จสมบูรณ์; * ถูกพUSH
$a - b * c/d$	*	* เสร็จสมบูรณ์; / ถูกพUSH
$a - b * c + d$	*	* และ — เสร็จสมบูรณ์; + ถูกพUSH

วงเล็บเป็นเพียงสิ่งที่เพิ่มความซับซ้อนเข้ามา เราสามารถมองวงเล็บเปิดเป็นเครื่องหมายที่มีลำดับความสำคัญสูงเมื่อมันเป็นสัญลักษณ์นำเข้า (เพื่อให้ตัวดำเนินการที่ค้างอยู่ ยังคงค้างอยู่ต่อไป) และเป็นตัวดำเนินการที่มีลำดับความ

สำคัญต่ำเมื่อมันอยู่ในสแต็ก (เพื่อไม่ให้มันถูกลบออกโดยบังเอิญจากตัวดำเนินการอื่น) วงเล็บปิดถูกจัดการแบบกรณีพิเศษ

เพื่อดูว่าอัลกอริทึมนี้ทำงานอย่างไร เราจะแปลงสมการอินพีชชันยาวด้านบนให้เป็นรูปแบบโพสต์ฟิกซ์ อันดับแรก สัญลักษณ์ a จะถูกอ่าน ดังนั้นมันจึงถูกส่งผ่านไปยังผลลัพธ์

จากนั้น $+$ จะถูกอ่านและpushลงในสแต็ก ถัดมา b จะถูกอ่านและส่งผ่านไปยังผลลัพธ์ สถานการณ์ ณ จุดนี้เป็นดังนี้:

$$\begin{array}{c|c} + & a \quad b \\ \hline \text{Stack} & \text{Output} \end{array}$$

ถัดมา อ่านพบ $*$ ข้อมูลบนสุดของสแต็กเครื่องหมายมีลำดับความสำคัญต่ำกว่า $*$ ดังนั้นจึงไม่มีอะไรถูกส่งออกไปที่ผลลัพธ์และ $*$ จะถูกpushลงในสแต็ก ถัดมา c จะถูกอ่านและเป็นผลลัพธ์ จนถึงตอนนี้ เรามีดังนี้

$$\begin{array}{c|c} * & a \quad b \quad c \\ \hline + & \\ \hline \text{Stack} & \text{Output} \end{array}$$

สัญลักษณ์ถัดไปคือ $+$ เมื่อตรวจสอบสแต็ก เราพบว่าเราจะต้องป๊อป $*$ และส่งออกไปที่ผลลัพธ์; จากนั้นป๊อป $+$ อีกตัว ซึ่งไม่ได้มีความสำคัญต่ำกว่าแต่เท่ากัน ไปที่ผลลัพธ์; และจากนั้นจึงpush $+$

$$\begin{array}{c|c} + & a \quad b \quad c \quad * \quad + \\ \hline \text{Stack} & \text{Output} \end{array}$$

สัญลักษณ์ถัดไปที่จะอ่านคือ $($ เนื่องจากมีความสำคัญสูงสุด มันจึงถูกวางลงในสแต็ก จากนั้น d จะถูกอ่านและแสดงเป็นผลลัพธ์

$$\begin{array}{c|c} (& a \quad b \quad c \quad * \quad + \quad d \\ \hline + & \\ \hline \text{Stack} & \text{Output} \end{array}$$

เราทำต่อโดยการอ่าน $*$ เนื่องจากวงเล็บเปิดจะไม่ถูกดึงออกยกเว้นเมื่อประมวลผลวงเล็บปิด จึงไม่มีข้อมูลออกไปที่ผลลัพธ์ ถัดมา e จะถูกอ่านและส่งออกเป็นผลลัพธ์

$$\begin{array}{c|c} * & a \quad b \quad c \quad * \quad + \quad d \quad e \\ (& \\ + & \\ \hline \text{Stack} & \text{Output} \end{array}$$

สัญลักษณ์ถัดไปที่อ่านพบคือ + เราป้อน * ออกมาที่ผลลัพธ์แล้วจึงพুষ + จากนั้นเราอ่านและส่ง f ไปที่ผลลัพธ์

$$\begin{array}{c} + \\ (\\ + \\ \hline \text{Stack} \end{array} \quad \begin{array}{c} a \quad b \quad c \quad * \quad + \quad d \quad e \quad * \quad f \\ \hline \text{Output} \end{array}$$

ตอนนี้เราอ่านเจอ) ดังนั้นสแต็กจะถูกทำให้ว่างเปล่าลงไปถึง (เราแสดง + ออกที่ผลลัพธ์

$$\begin{array}{c} + \\ \hline \text{Stack} \end{array} \quad \begin{array}{c} a \quad b \quad c \quad * \quad + \quad d \quad e \quad * \quad f \quad + \\ \hline \text{Output} \end{array}$$

เราอ่าน * ถัดไป; มันถูกพুষลงในสแต็ก จากนั้น g จะถูกอ่านและแสดงผลลัพธ์

$$\begin{array}{c} * \\ + \\ \hline \text{Stack} \end{array} \quad \begin{array}{c} a \quad b \quad c \quad * \quad + \quad d \quad e \quad * \quad f \quad + \quad g \\ \hline \text{Output} \end{array}$$

ตอนนี้ข้อมูลนำเข้าว่างเปล่าแล้ว ดังนั้นเราจึงป้อนและส่งสัญลักษณ์จากสแต็กออกมาที่ผลลัพธ์จนกว่าสแต็กจะว่างเปล่า

$$\begin{array}{c} \hline \text{Stack} \end{array} \quad \begin{array}{c} a \quad b \quad c \quad * \quad + \quad d \quad e \quad * \quad f \quad + \quad g \quad * \quad + \\ \hline \text{Output} \end{array}$$

เช่นเคย การแปลงนี้ใช้เวลาเพียง $O(N)$ และทำงานในการอ่านผ่านข้อมูลนำเข้าเพียงรอบเดียว เราสามารถเพิ่มการลบและการหารลงในชุดตัวดำเนินการนี้ได้โดยกำหนดให้การลบและการบวกมีความสำคัญเท่ากัน และการคูณและการหารมีความสำคัญเท่ากัน จุดที่ละเอียดอ่อนคือสมการ $a - b - c$ จะถูกแปลงเป็น $a \ b \ - \ c \ -$ ไม่ใช่ $a \ b \ c \ -$ — อัลกอริทึมของเราทำงานได้อย่างถูกต้องเนื่องจากตัวดำเนินการเหล่านี้มีคุณสมบัติการจัดกลุ่มจากซ้ายไปขวา (associate from left to right) ซึ่งอาจไม่เป็นเช่นนี้เสมอไปในกรณีทั่วไป ตัวอย่างเช่น การยกกำลังจะจัดกลุ่มจากขวามาซ้าย: $2^{2^3} = 2^8 = 256$ ไม่ใช่ $4^3 = 64$ เราถึงปัญหาการเพิ่มเครื่องหมายยกกำลังลงในชุดตัวดำเนินการไว้เป็นแบบฝึกหัด

การเรียกใช้ฟังก์ชัน (Function Calls)

อัลกอริทึมในการตรวจสอบความสมดุลของสัญลักษณ์ นำเสนอแนวทางในการนำไปใช้กับการเรียกฟังก์ชันในภาษาคอมไพเลอร์เชิงกระบวนการ (compiled procedural) และเชิงวัตถุ (object-oriented) ปัญหาในที่นี้คือ เมื่อมีการเรียกฟังก์ชันใหม่ ตัวแปรทั้งหมดที่จำเพาะเจาะจง (local variables) ต่อ routine ผู้เรียก จำเป็นต้องได้รับการบันทึกโดยระบบ เนื่องจากมีฉะนั้นฟังก์ชันใหม่จะเขียนกับหน่วยความจำที่ใช้โดยตัวแปรของ routine ผู้เรียก นอกจากนี้ ตำแหน่งปัจจุบันใน routine จะต้องถูกบันทึกไว้...